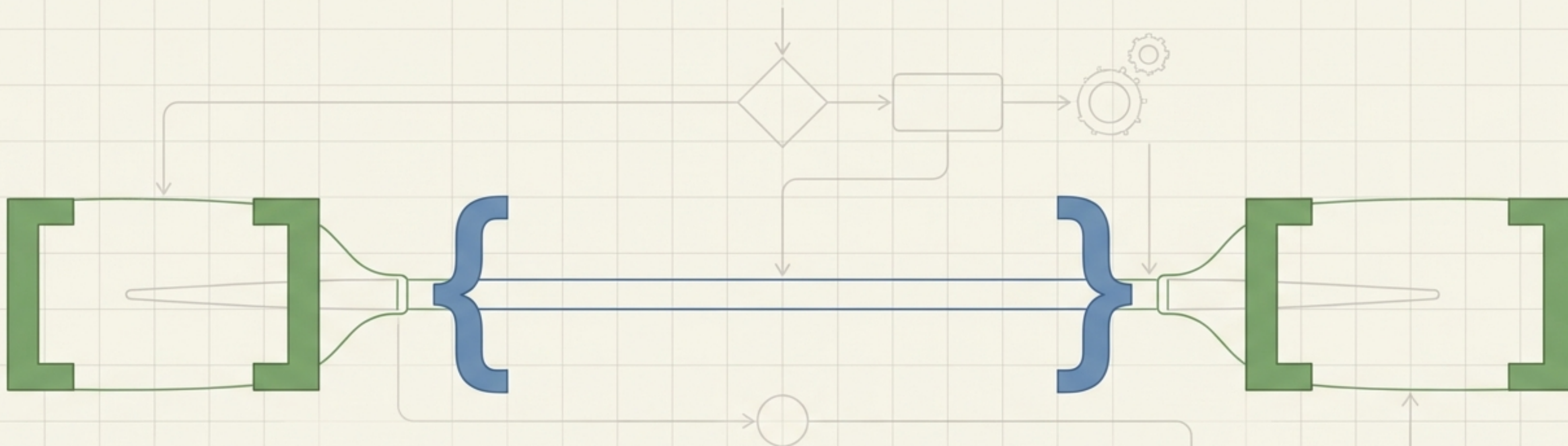
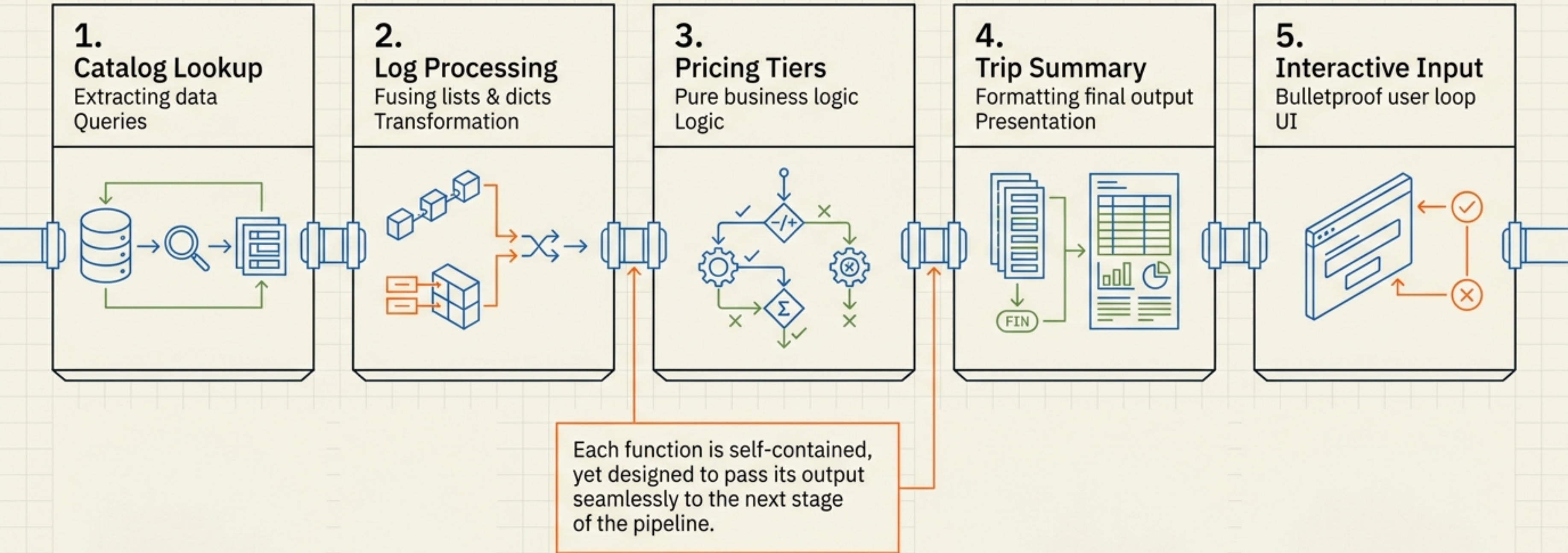




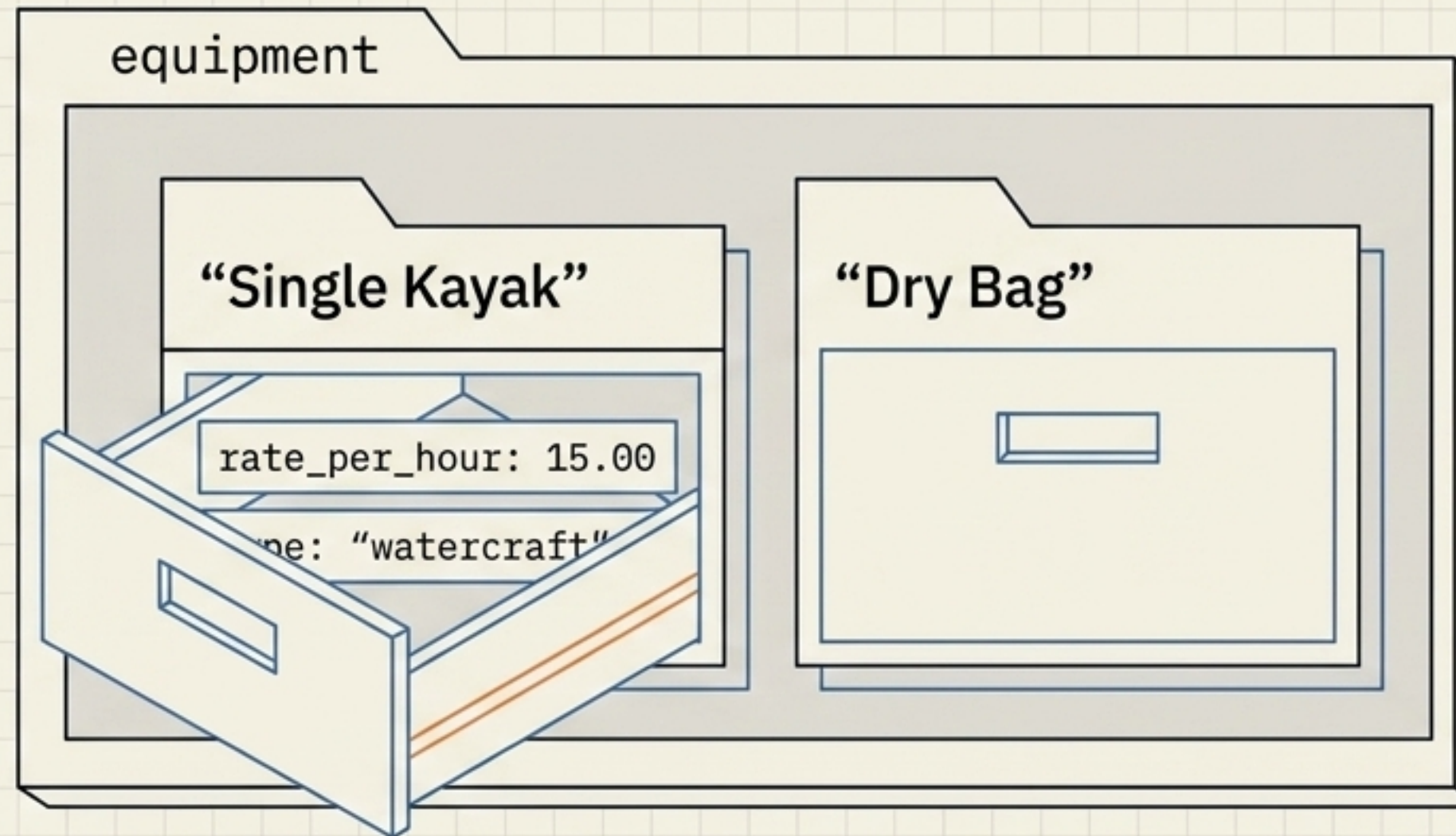
Building the Data River

Tracing the flow of data from raw dictionaries to dynamic user receipts in the Riverside Kayak Python Skill Builder.

Four Isolated Challenges. One Continuous Flow of Data.



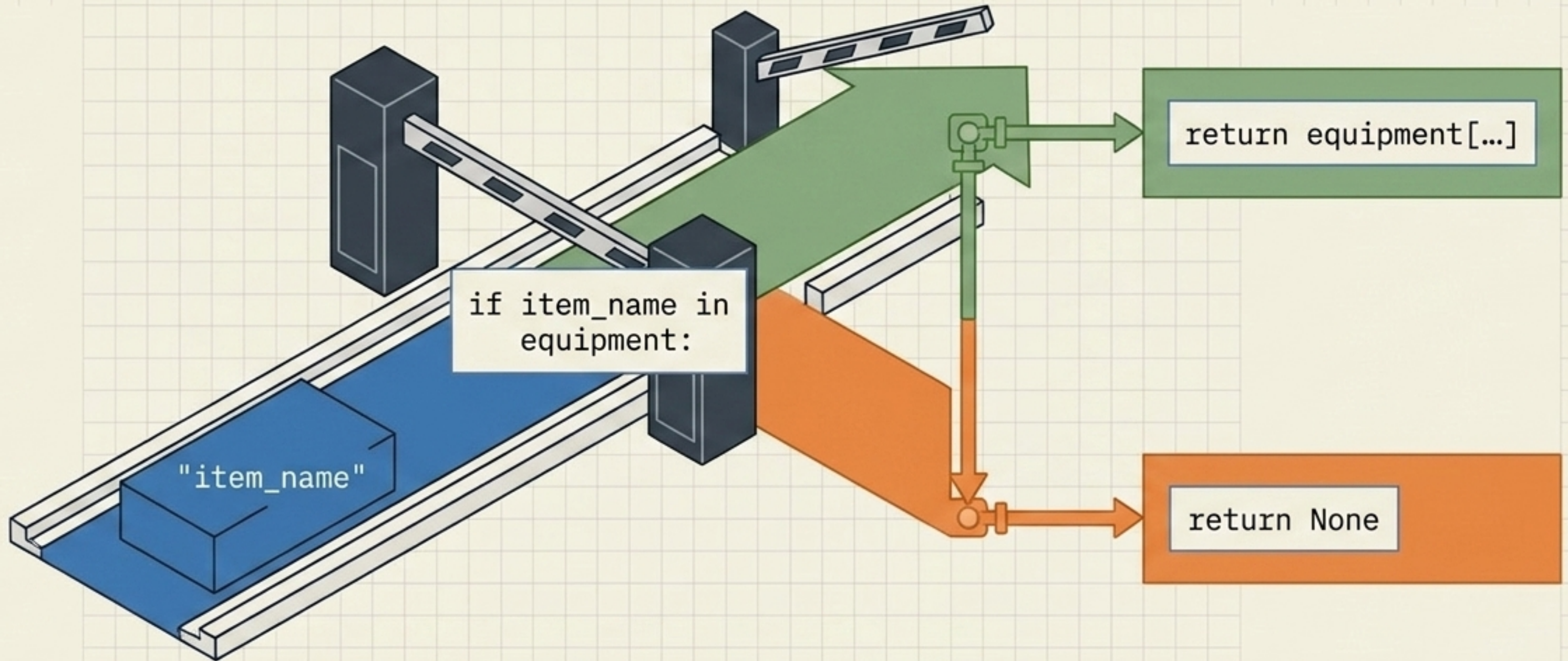
The Database: Structuring Truth with Nested Dictionaries



```
equipment[item_name]["rate_per_hour"]
```

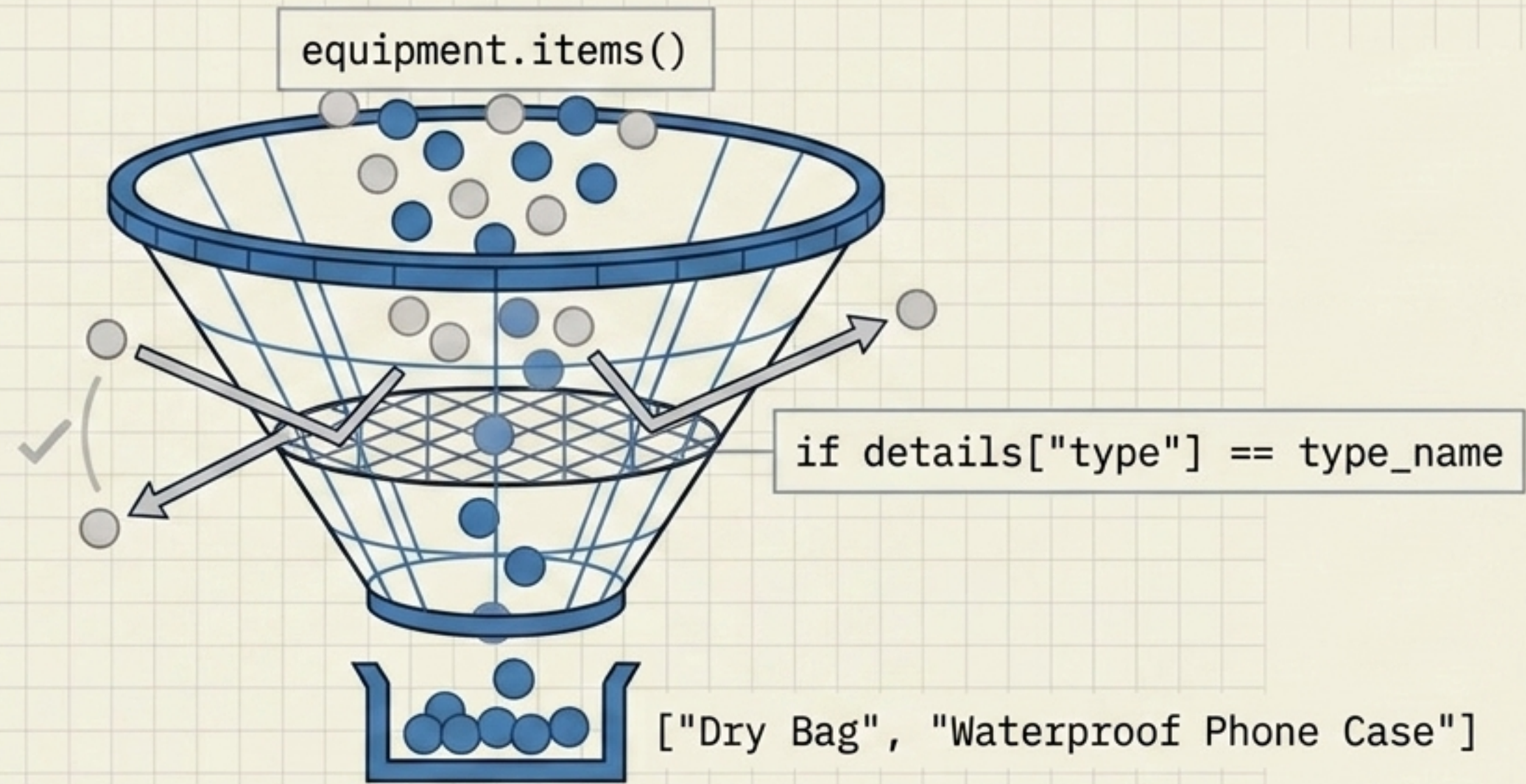
Grouping related **attributes** under a single string key allows for $O(1)$ instantaneous lookups across the entire program, bypassing the need to search through lists.

Defending Against KeyErrors with Guard Clauses



The `get_rate()` function uses a 'Look Before You Leap' pattern. Returning `None` instead of crashing allows downstream functions to handle missing items gracefully without halting the system.

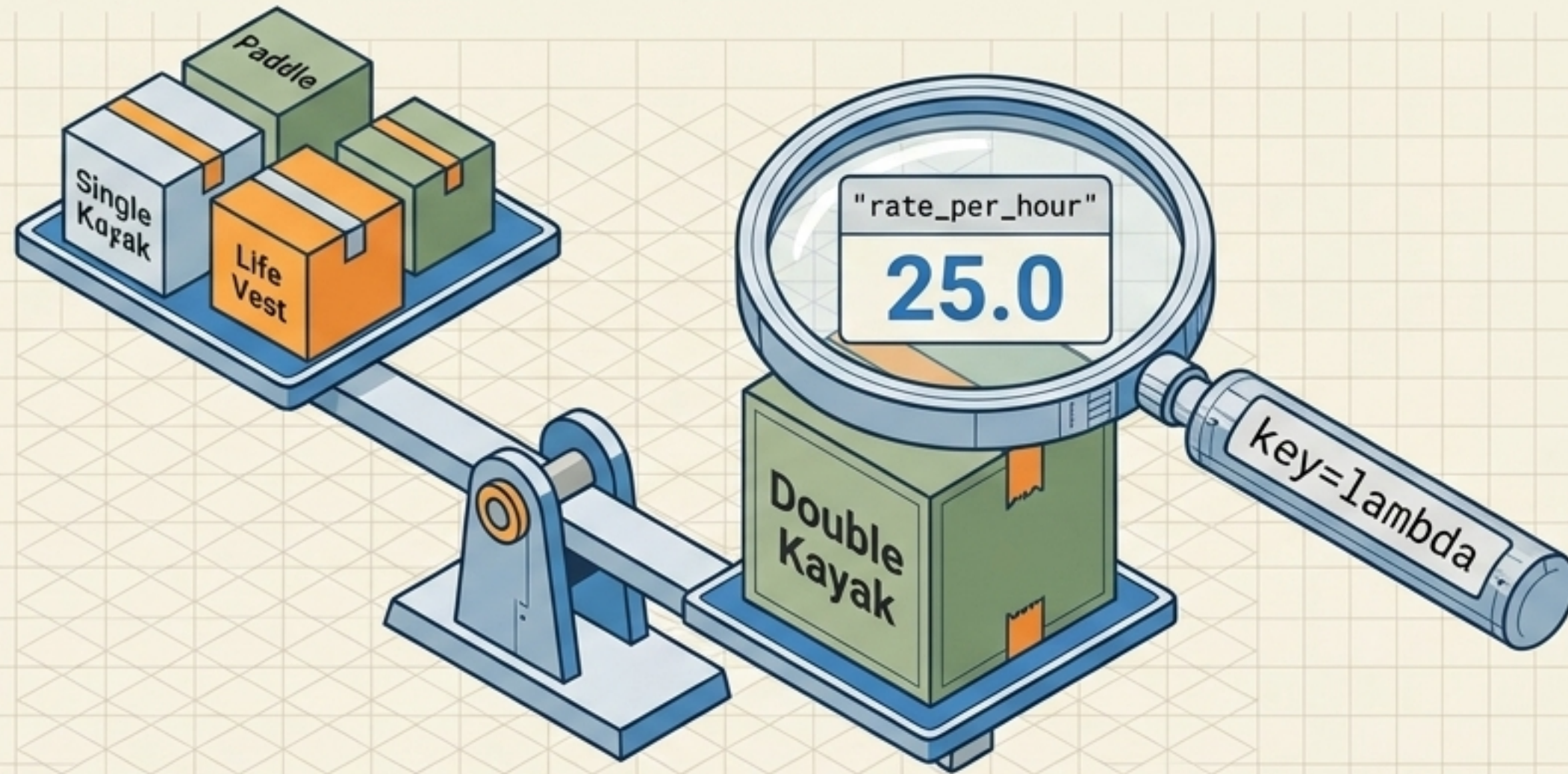
The Filter Engine: Condensing Logic with List Comprehensions



```
[name for name, details in equipment.items() if ...]
```

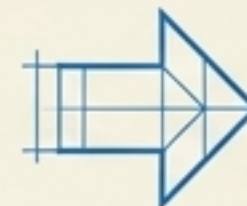
A 4-line for-loop collapsed into a single, highly readable expression of intent.

Finding Extremes Using Anonymous Functions



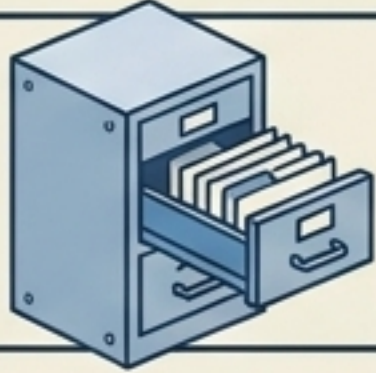
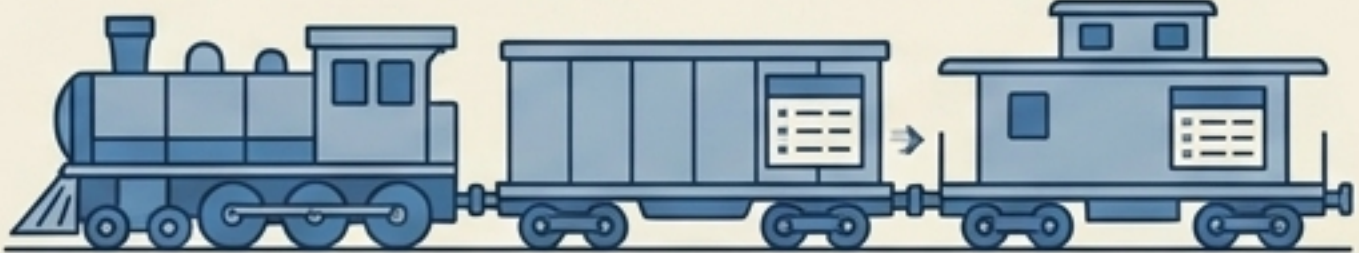
```
max(equipment, key=lambda item_name: equipment[item_name]["rate_per_hour"])
```

`max()` normally sorts dictionaries by alphabetical keys.
The lambda function intercepts this process, instructing Python:
"Ignore the item name; weigh the nested price."



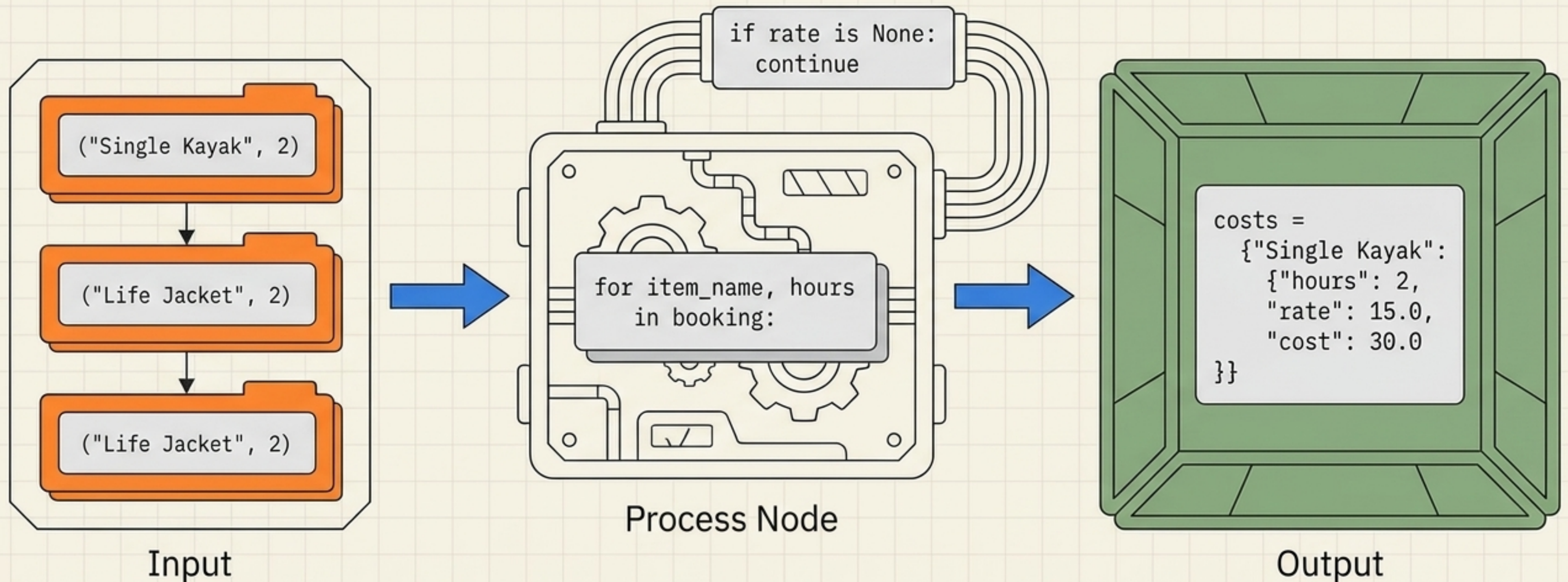
("Double Kayak",	25.0)
------------------	-------

Contrasting Data Shapes: Catalogue vs. Log

The Catalogue (Dictionary)	The Booking Log (List of Tuples)
	
Format: <code>{"Item": {"rate": 15.0}}</code>	Format: <code>[("Item", 2)]</code>
Purpose: Master Reference	Purpose: Sequential Record of events
Access: By Key (Name)	Access: By Iteration (for loop)
Mutable: Yes (Values can update)	Mutable: Locked (Tuples are immutable)

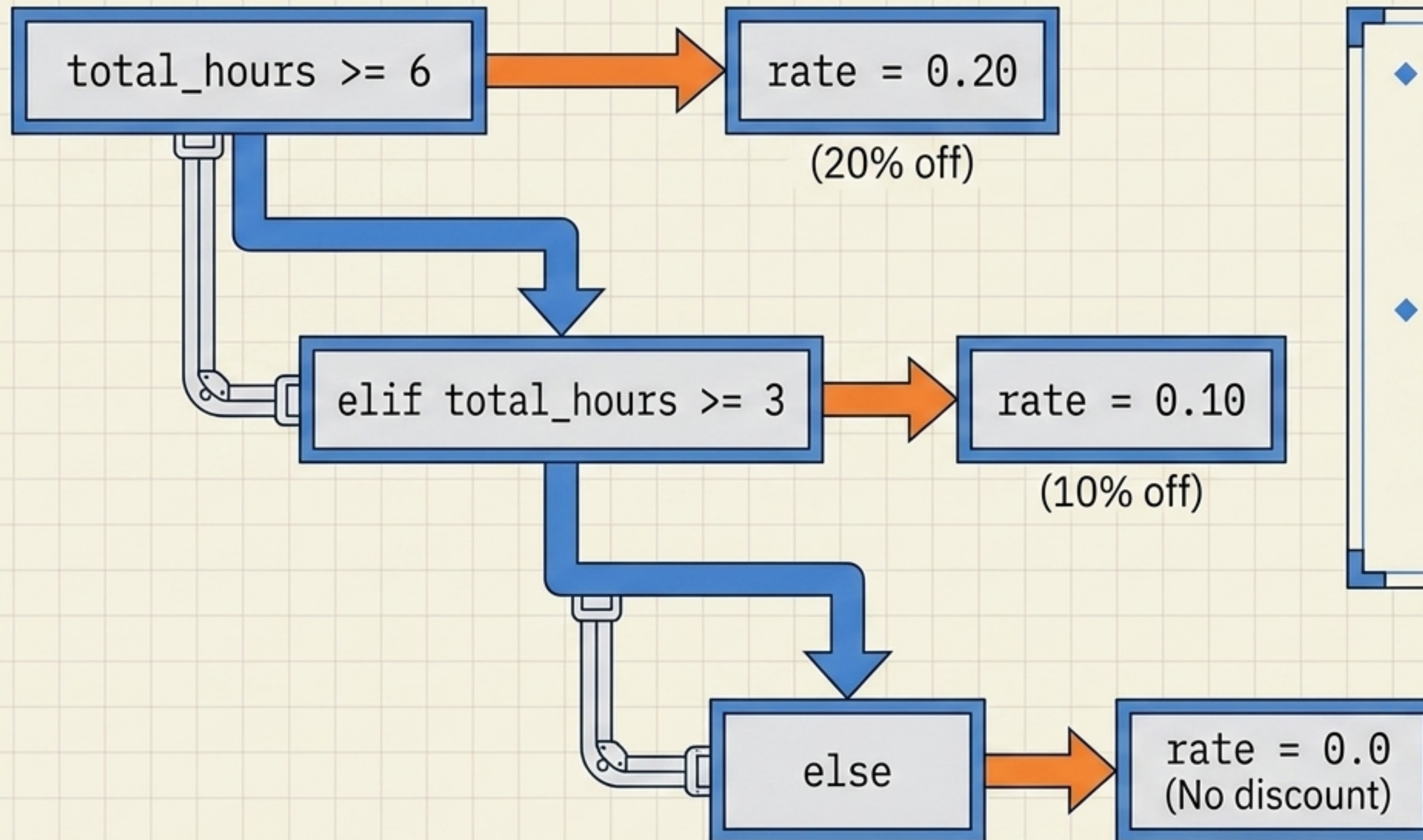
Challenge 2 bridges these structures—transforming the rigid booking log into a dynamic, calculated dictionary.

The Great Transformation: Fusing Lists into Dictionaries



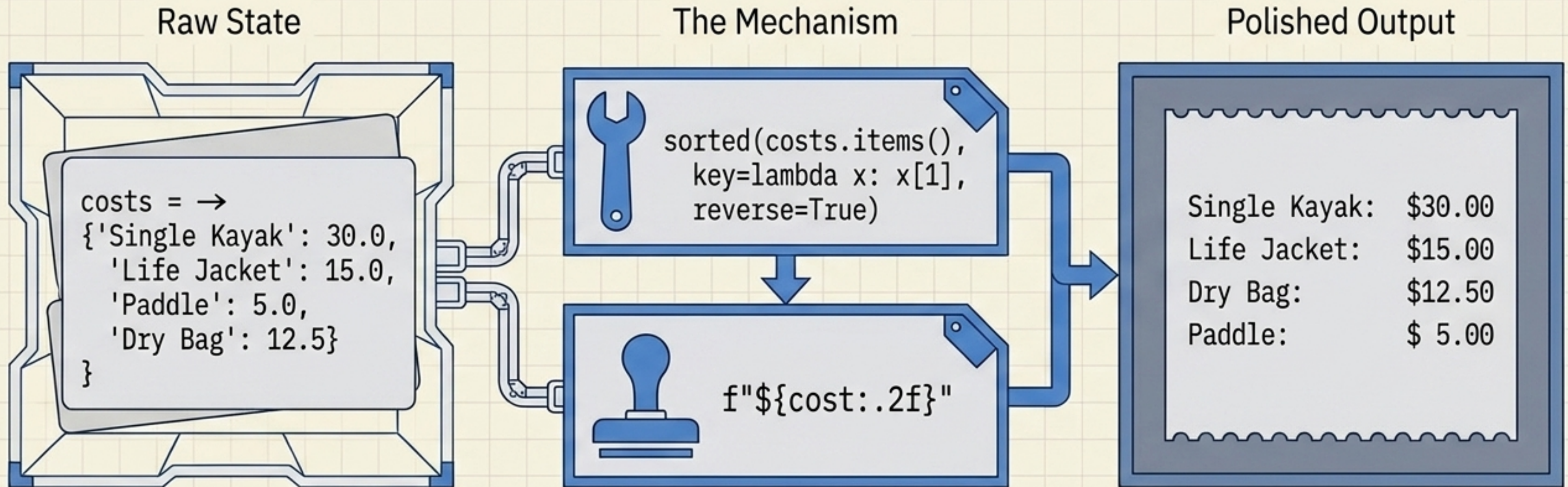
Tuple unpacking seamlessly separates `item_name` and `hours`, allowing the script to enrich raw user logs with live financial data.

The Logic Cascade: Pure Functions and Tiered Pricing



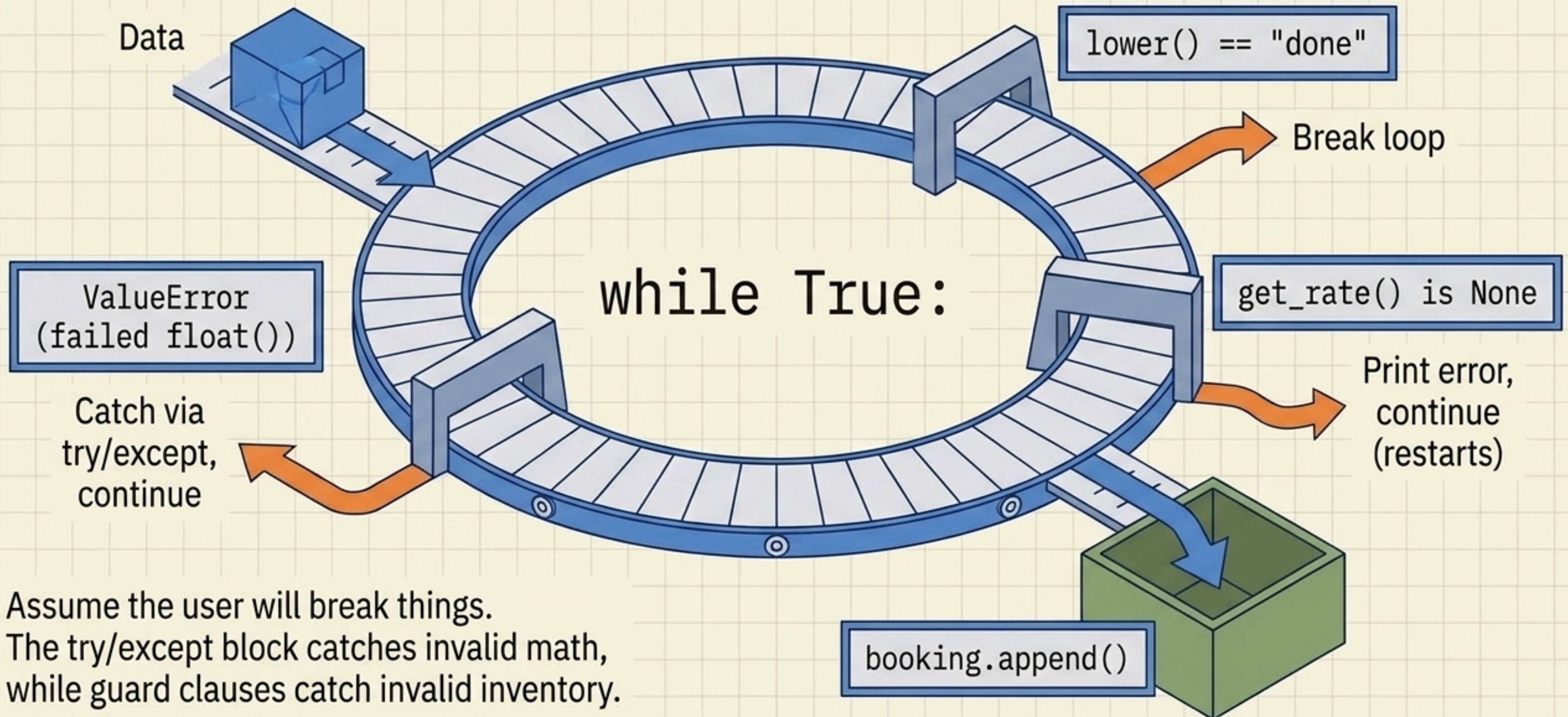
- ◆ **Pure Function:** Relies entirely on its inputs (`total_hours`, `subtotal`). No hidden state.
- ◆ **Currency Formatting:** Uses `round(..., 2)` to eliminate floating-point artifacts (e.g., preventing \$6.0000000001).

The Presentation Layer: Sorting and f-String Formatting

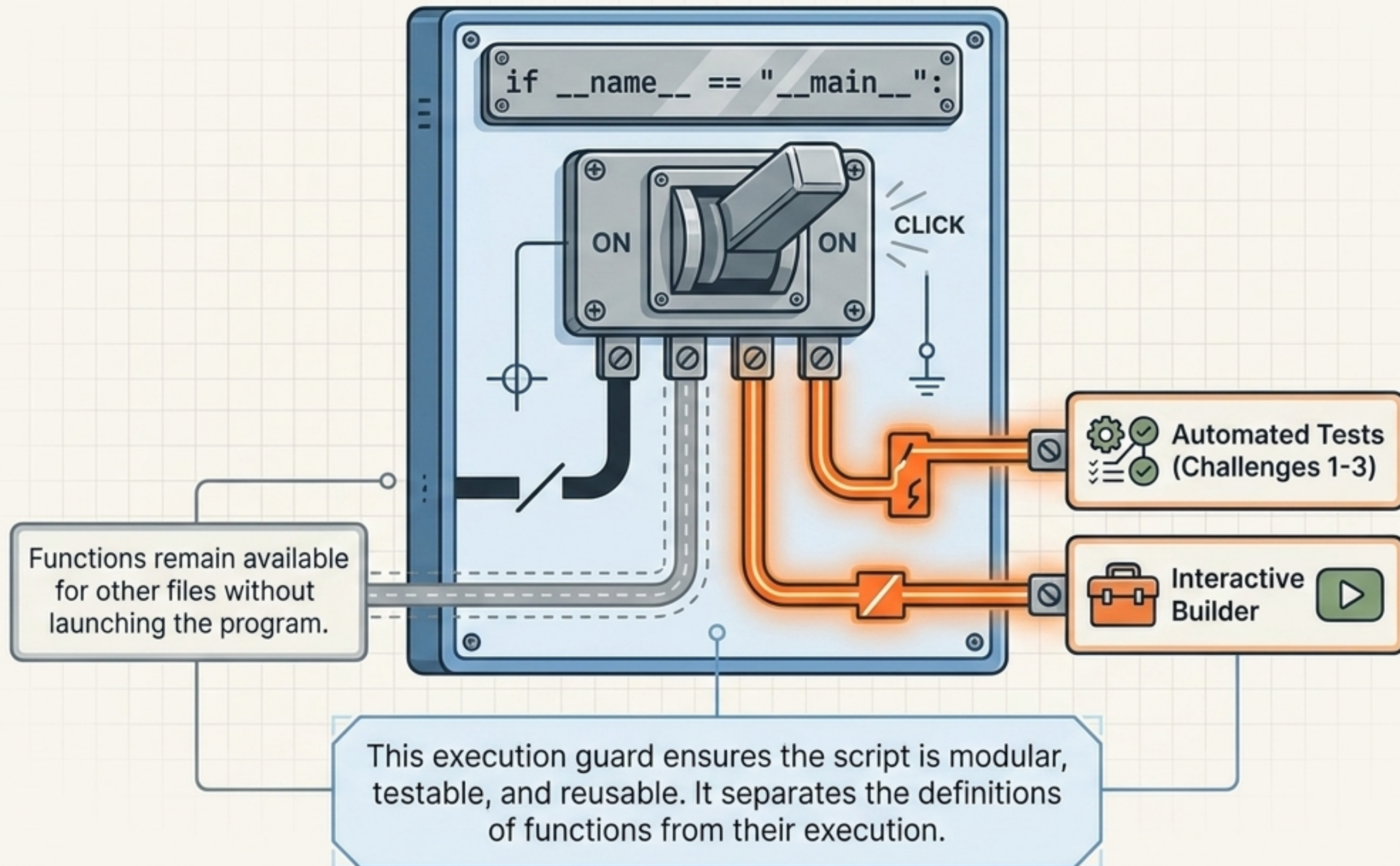


Generating a trip summary requires sorting by generated tuple lists (`.items()`) and forcing strict currency displays (`:.2f`) to ensure professional output.

The Defensive Loop: Bulletproofing the User Interface

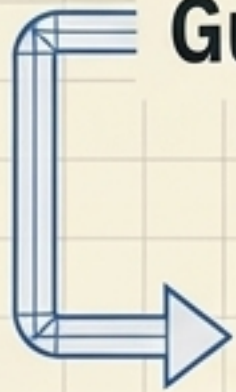


The Orchestrator: Safe Script Execution



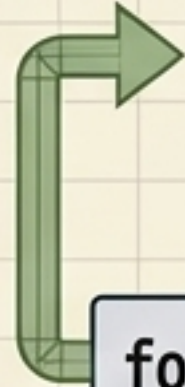
Synthesis: Mapping App Logic to Universal Design Patterns

Guard Clauses



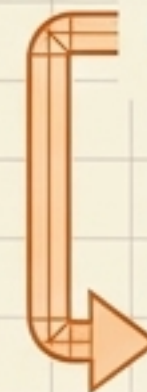
```
get_rate()  
Interactive Loop
```

Tuple Unpacking



```
for item, hours in booking
```

Generator Expressions



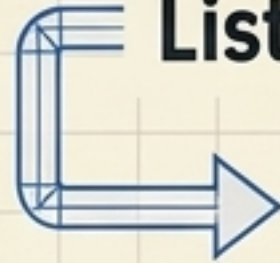
```
sum(details["cost"] for...)
```

Anonymous Functions



```
max(key=lambda)
```

List Comprehensions



```
[name for name in equipment]
```

Exception Handling



```
try:  
    float()  
except ValueError:
```

You aren't just building a kayak rental script; you are deploying professional-grade Python architecture that scales to any data processing pipeline.